BỘ GIÁO DỤC VÀ ĐÀO TẠO
HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
CƠ SỞ TẠI TP. HỒ CHÍ MINH
KHOA KỸ THUẬT ĐIỆN TỬ 2
NGÀNH: CÔNG NGHỆ KỸ THUẬT ĐIỆN, ĐIỆN TỬ

BÁO CÁO ĐỒ ÁN HỌC PHẦN

MÔN: CẤU TRÚC DỮ LIỆU VÀ GIẢI THUẬT

Mã học phần: INT13185 — Nhóm 03

ĐỀ TÀI:

KHOẢNG CÁCH CHỈNH SỬA VÀ DÃY PHÉP BIẾN ĐỔI

Mã đề: DT056

Giảng viên hướng dẫn : **Hồ Nhựt Minh**

Sinh viên thực hiện : **NGUYỄN NGƯ NHỰT**

MSSV : **N24DCDT065**

Lớp : **D24CQDT01-N**

TP. HCM, tháng 10 năm 2026

PHIẾU GIAO ĐỀ TÀI

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG — CƠ SỞ TẠI TP. HỒ CHÍ MINH

Học phần INT13185 — Nhóm 03 — Giảng viên hướng dẫn: Hồ Nhựt Minh

Mã đề: DT056

Tên đề: Khoảng cách chỉnh sửa và dãy phép biến đổi

Sinh viên: Nguyễn Ngư Nhựt — MSSV N24DCDT065 — Lớp D24CQDT01-N

Căn cứ: Chương 4, Bài tập chương 4 câu 35, trang in 112 (trang PDF 124).

ĐỀ BÀI

Viết chương trình biến một xâu thành xâu khác bằng chèn, xóa hoặc thay một ký tự, sao cho dùng ít thao tác nhất. Mỗi thao tác có chi phí 1, giữ nguyên ký tự giống nhau không mất chi phí. Sinh viên lập bảng theo hai tiền tố, truy vết và kiểm rằng thực hiện các bước đã chọn thật sự tạo ra xâu đích.

DỮ LIỆU VÀO

Nhập hoặc khai báo X, Y bằng chữ cái Latin không dấu, độ dài mỗi xâu 0 đến 30. Xử lý theo ký tự, phân biệt hoa thường. Không có thao tác đổi chỗ hai ký tự và không có chi phí trọng số khác nhau.

KẾT QUẢ RA

In số thao tác nhỏ nhất và một dãy chèn, xóa, thay có mô tả rõ. Có thể in các xâu sau từng bước để kiểm kết quả.

HAI VÍ DỤ

Ví dụ 1: X = "CAT", Y = "CUT". Hai ký tự C, T giữ nguyên, chỉ cần thay ký tự giữa nên khoảng cách bằng 1.

Ví dụ 2: X = "A", Y = "". Xâu đích rỗng nên xóa ký tự duy nhất của xâu nguồn, khoảng cách bằng 1.

NĂM BƯỚC THỰC HIỆN

1. Đặt f[i, j] là số thao tác ít nhất biến i ký tự đầu X thành j ký tự đầu Y.
2. Gán hàng 0 bằng 0, 1, …, n và cột 0 bằng 0, 1, …, m để biểu diễn chèn hoặc xóa toàn bộ.
3. Nếu ký tự bằng nhau lấy ô chéo; nếu khác, lấy 1 cộng nhỏ nhất trong ba khả năng chèn, xóa, thay.

4. Lưu hoặc nhận ra bước trước khi truy ngược; đảo các bước để có thứ tự biến đổi từ X đến Y.
5. Quy định cách mô tả vị trí khi xâu thay đổi, thực hiện lại dãy thao tác và kiểm xâu cuối bằng Y.

BA HÀM NÊN VIẾT

- khoang_cach_sua(X, Y)
- truy_vet_sua(X, Y, f)
- ap_dung_cac_buoc(X, buoc)

NĂM CA CẦN KIỂM TRA

1. X = "", Y = "": khoảng cách 0, danh sách thao tác rỗng.
2. X = "", Y = "AB": khoảng cách 2 bằng hai lần chèn.
3. X = "ABC", Y = "ABC": khoảng cách 0, không thay ký tự giống nhau.
4. X = "AB", Y = "BA": khoảng cách 2, không được dùng đổi chỗ một lần.
5. X = "ABC", Y = "AC": khoảng cách 1 bằng xóa B.

PHÂN TÍCH VÀ NHẬN XÉT

Với độ dài m, n, bảng có (m+1)(n+1) ô, mỗi ô xét tối đa ba lựa chọn nên thời gian và bộ nhớ O(mn). Truy vết tối đa m+n bước. Phải phân biệt độ dài dãy thao tác với số ô của bảng; nhiều dãy sửa tối ưu khác nhau vẫn được chấp nhận.

YÊU CẦU THÊM SAU KHI PHẦN CƠ BẢN CHẠY ĐÚNG

Tùy chọn: in nguyên bảng với nhãn hai xâu cho trường hợp độ dài tối đa 6 để giải thích các bước truy vết.

SẢN PHẨM

Mã nguồn, các kết quả chạy ví dụ và ca kiểm tra, một lần vết bằng tay, báo cáo theo mẫu môn học. Chạy từ bàn phím hoặc truyền trực tiếp các giá trị vào hàm.

PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG

Dùng rubric chung của học phần và hướng dẫn minh chứng đúng mã đề trong tệp 00_HUONG_DAN_SINH_VIEN.html. Vị trí bằng chứng của từng tiêu chí ghi ở cột cuối Bảng 0.1.

Bảng 0.1. Sáu tiêu chí, trọng số và vị trí minh chứng trong báo cáo

Tiêu chí	**Trọng số**	**Nội dung chấm**	**Minh chứng trong báo cáo**	GV1	GV2	Quy đổi
C1	1,5	Đặc tả bài toán, đầu vào, đầu ra	Mục 1.3, Bảng 1.1 và 1.2, mục 3.1			
C2	2,0	Biểu diễn dữ liệu và phần tự cài	Mục 2.2, Bảng 2.1, mục 3.2			
C3	2,5	Tính đúng và lần vết thuật toán	Mục 2.3, 3.3, 3.4, 3.5			
C4	2,0	Kiểm thử, đối chiếu và độ phức tạp	Mục 3.6, 4.2 đến 4.8			
C5	1,0	Báo cáo, minh chứng, kho mã	Chương 4, Phụ lục A và B			
C6	1,0	Bảo vệ và giải thích sản phẩm	Toàn bộ báo cáo và phần tự kiểm			

Điểm mỗi tiêu chí chấm theo thang 0–10, quy đổi bằng điểm × trọng số / 10, cộng sáu dòng rồi làm tròn một chữ số thập phân. Hai người chấm lấy trung bình từng tiêu chí; nếu chỉ một người chấm thì ghi rõ.

Tổng điểm: ...../10

Nhận xét của giảng viên:

.....

.....

.....

Người chấm 1 (ký, họ tên): .....

Người chấm 2 (ký, họ tên): .....

LỜI CAM ĐOAN

Em xin cam đoan báo cáo này là kết quả làm việc của cá nhân em trong học phần Cấu trúc dữ liệu và Giải thuật, mã đề DT056. Toàn bộ mã nguồn, bộ kiểm thử, số liệu và hình ảnh trong báo cáo do em tự thực hiện trên máy của mình.

Các phần tham khảo đều được ghi nguồn: công thức quy hoạch động lấy từ giáo trình INT1306 chương 4; phần cài đặt, kiểm thử và các bảng kết quả do em tự làm. Em không sao chép bài của sinh viên khác và không sử dụng kết quả chưa chạy thật.

Nếu phát hiện nội dung không trung thực, em xin chịu trách nhiệm theo quy định của học viện.

TP. Hồ Chí Minh, tháng 10 năm 2026

Sinh viên thực hiện

(Ký và ghi rõ họ tên)

NGUYỄN NGƯ NHỰT

LỜI CẢM ƠN

Trước hết em cảm ơn thầy Hồ Nhựt Minh đã giao đề tài DT056 và hướng dẫn cách lập bảng theo hai tiền tố, cách truy vết và cách kiểm lại dãy thao tác. Những buổi trao đổi trên lớp giúp em hiểu rõ vì sao phải phân biệt vị trí trên xâu gốc với vị trí trên xâu đang thay đổi.

Em cảm ơn quý thầy cô khoa Kỹ thuật Điện tử 2 đã truyền đạt phần lý thuyết quy hoạch động và kỹ năng kiểm thử, nhờ đó em biết đặt kỳ vọng trước khi chạy và biết đọc bảng truy vết thay vì chỉ nhìn con số cuối.

Bài làm còn hạn chế vì thời gian có hạn, em mong nhận được góp ý để hoàn thiện.

TÓM TẮT

Báo cáo trình bày bài toán khoảng cách chỉnh sửa giữa hai xâu: biến xâu X thành xâu Y bằng ít thao tác chèn, xóa, thay nhất, mỗi thao tác chi phí 1, giữ nguyên ký tự giống nhau không mất chi phí. Ngoài con số khoảng cách, bài làm còn phải sinh ra dãy thao tác cụ thể, thực hiện lại dãy đó và kiểm xâu cuối đúng bằng Y.

Cách làm dựa trên quy hoạch động hai tiền tố: bảng f có (m+1)(n+1) ô, mỗi ô lấy 1 cộng giá trị nhỏ nhất của ô trên, ô trái và ô chéo; nếu hai ký tự cuối bằng nhau thì lấy nguyên ô chéo. Sau khi tính xong, truy ngược từ ô cuối về ô gốc để thu dãy thao tác rồi đảo lại. Điểm riêng của bài làm là quy ước vị trí thao tác tính trên xâu đang biến đổi, nhờ vậy hàm ap_dung_cac_buoc chạy lại được nguyên dãy và tự kiểm được kết quả.

Chương trình viết bằng Julia 1.13.1, chỉ dùng thư viện chuẩn, gồm ba hàm bắt buộc theo phiếu và hai hàm hỗ trợ kiểm tra dữ liệu, in bảng. Bộ kiểm thử có 13 ca độc lập (hai ví dụ, năm ca phiếu, hai ca tự thiết kế, ba ca biên, một ca dữ liệu sai) và 400 cặp xâu ngẫu nhiên để kiểm bất biến; tổng 21.803 phép kiểm, tất cả đạt.

Kết quả đo trên ba quy mô m = n = 10, 20, 30 cho thấy số ô bảng tăng đúng theo (m+1)(n+1), thời gian tăng theo số ô, phù hợp với phân tích O(mn). Báo cáo cũng nêu giới hạn của bản cài (chi phí đơn vị, không có đổi chỗ, bộ nhớ O(mn) do phải giữ bảng để truy vết) và hướng phát triển tiếp theo.

MỤC LỤC

PHIẾU GIAO ĐỀ TÀI **1**
PHIẾU CHẤM VÀ ĐỐI CHIẾU MINH CHỨNG **3**
LỜI CAM ĐOAN **4**
LỜI CẢM ƠN **5**
TÓM TẮT **6**
DANH MỤC BẢNG **9**
DANH MỤC HÌNH **10**
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI **11**
1.1. Bài toán và lý do thực hiện 11
1.2. Mục tiêu và sản phẩm 11
1.3. Phạm vi, đầu vào và đầu ra 12
1.3.1. Miền và quy ước đầu vào 12
1.3.2. Kết quả và trường hợp đặc biệt 12
1.4. Cách thực hiện và cấu trúc báo cáo 13
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT **14**
2.1. Khái niệm và mô hình dữ liệu 14
2.2. Cấu trúc dữ liệu sử dụng 14
2.2.1. Dữ liệu và thao tác chính 14
2.2.2. Quy ước và điều kiện hợp lệ 15
2.3. Nguyên lý thuật toán 15
2.3.1. Ý tưởng quy hoạch động 15
2.3.2. Khởi tạo và kết thúc 15
2.3.3. Truy vết và quy ước vị trí 16
2.4. Cơ sở đánh giá thời gian và bộ nhớ 16
CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT **17**
3.1. Phân tích yêu cầu chương trình 17
3.1.1. Hợp đồng input/output 17
3.1.2. Kiểm tra đầu vào 17
3.2. Thiết kế các hàm và dữ liệu 17
3.3. Thuật toán và mã nguồn chính 18
3.3.1. Giả mã theo cài đặt 18
3.3.2. Đoạn mã và giải thích 19
3.3.3. Xử lý tình huống đặc biệt 19
3.4. Lần vết ví dụ nhỏ 20
3.5. Giải thích tính đúng và điều kiện dừng 21

3.5.1. Điều kiện khởi đầu và duy trì 21
3.5.2. Kết thúc và output 21
3.6. Phân tích độ phức tạp 22
CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT 23
4.1. Môi trường và kế hoạch kiểm thử 23
4.2. Bảng kết quả 13 ca kiểm thử 23
4.3. Hai ví dụ trên phiếu 25
4.4. Hai ca sinh viên tự thiết kế 26
4.5. Ba ca biên phù hợp với đề tài 27
4.6. Kiểm tra input không hợp lệ — LOI01 29
4.7. Kết quả toàn bộ kiểm thử 30
4.8. Khảo sát ba quy mô 31
4.9. Kho mã và trạng thái công bố 31
CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN 34
5.1. Mức hoàn thành so với phiếu 34
5.2. Giới hạn và bài học 34
5.3. Hướng phát triển 34
TÀI LIỆU THAM KHẢO 36
PHỤ LỤC A. TỔ CHỨC MÃ VÀ HƯỚNG DẪN CHẠY 37
PHỤ LỤC B. DANH MỤC BẰNG CHỨNG 38
PHỤ LỤC C. TỰ KIỂM TRƯỚC KHI NỘP 40
PHỤ LỤC D. HƯỚNG DẪN NỘP BÀI 41

DANH MỤC BẢNG

Bảng 0.1. Sáu tiêu chí, trọng số và vị trí minh chứng trong báo cáo 3
Bảng 1.1. Đặc tả dữ liệu vào 12
Bảng 1.2. Đặc tả kết quả đầu ra 12
Bảng 1.3. Đối chiếu yêu cầu của phiếu và sản phẩm 13
Bảng 2.1. Biến cấu trúc và điều kiện phải duy trì 15
Bảng 3.1. Hợp đồng của các hàm 17
Bảng 3.2. Tổ chức các hàm và tệp 18
Bảng 3.3. Tiền điều kiện và hậu điều kiện 18
Bảng 3.4. Bảng f cho CAT → CUT 20
Bảng 3.5. Lần vết CAT → CUT 20
Bảng 3.6. Lần vết SUNDAY → SATURDAY 20
Bảng 3.7. Chi phí của từng pha trong bản cài 22
Bảng 4.1. Ánh xạ năm ca kiểm trên phiếu 23
Bảng 4.2. Thiết kế 13 ca kiểm thử độc lập 24
Bảng 4.3. Kết quả thật và vị trí minh chứng 24
Bảng 4.4. Khảo sát ba quy mô 31
Bảng 4.5. Kho mã và trạng thái 31
Bảng 5.1. Mức hoàn thành và bằng chứng 34
Bảng PL.1. Các lệnh chạy 37
Bảng PL.2. Ảnh và log tương ứng 38
Bảng PL.3. Bốn tệp cần nộp 41

DANH MỤC HÌNH

Hình 4.1. A01 — Ví dụ 1: CAT → CUT, khoảng cách 1 25

Hình 4.2. A02 — Ví dụ 2: A → xâu rỗng, khoảng cách 1 25

Hình 4.3. A03 — Ca TK01: SUNDAY → SATURDAY, khoảng cách 3 26

Hình 4.4. A04 — Ca TK02: kitten → sitting, khoảng cách 3 27

Hình 4.5. A05 — Ca B01: A → B, khoảng cách 1 28

Hình 4.6. A06 — Ca B02: hai xâu 30 ký tự, khoảng cách 1 28

Hình 4.7. A07 — Ca B03: AAAA → AA, khoảng cách 2 29

Hình 4.8. A08 — Ca LOI01: dữ liệu sai bị từ chối 30

Hình 4.9. A09 — Tổng kết 13 ca, 21.803 phép kiểm đạt 30

Hình 4.10. A10 — Clone kho ở commit S rồi chạy lại bộ kiểm thử 32

Hình 4.11. A11 — Trang commit b77199d trên GitHub 33

Hình PL.1. A03P — Ảnh phụ: phần cuối lượt chạy julia main.jl phieu (ca P02–P05, tổng kết 5/5 đạt) 39

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Bài toán và lý do thực hiện

Khoảng cách chỉnh sửa giữa hai xâu là số thao tác ít nhất cần dùng để biến xâu nguồn X thành xâu đích Y, với ba thao tác được phép: chèn một ký tự, xóa một ký tự và thay một ký tự bằng ký tự khác. Mỗi thao tác tính chi phí 1; nếu hai ký tự đang xét giống nhau thì giữ nguyên và không mất chi phí. Bài toán không cho phép đổi chỗ hai ký tự và không có chi phí riêng cho từng loại thao tác.

Bài toán này xuất hiện ở nhiều chỗ quen thuộc: kiểm tra chính tả gợi ý từ gần đúng, so sánh hai phiên bản của một tệp văn bản, sửa lỗi nhận dạng ký tự, so khớp chuỗi sinh học. Điểm chung là cần biết hai xâu khác nhau bao nhiêu và khác ở đâu, chứ không chỉ cần một con số.

Bài toán gắn với tên Levenshtein [2]; cách lập bảng theo hai tiền tố thường được gọi là thuật toán Wagner–Fischer [3] và được trình bày trong giáo trình [1].

Phiếu DT056 nhấn vào phần thứ hai đó: ngoài khoảng cách nhỏ nhất, chương trình còn phải in ra dãy thao tác cụ thể và chứng minh bằng cách thực hiện lại dãy thao tác rồi so xâu cuối với Y. Yêu cầu này buộc phải lưu bảng quy hoạch động để truy vết, không thể chỉ giữ hai hàng như khi tính khoảng cách đơn thuần.

1.2. Mục tiêu và sản phẩm

Mục tiêu của bài làm gồm bốn phần, mỗi phần đều có điều kiện kiểm được:

- Tính đúng khoảng cách chỉnh sửa nhỏ nhất cho mọi cặp xâu trong miền $0 \le |X|, |Y| \le 30$, ký tự A–Z và a–z, phân biệt hoa thường.
- Sinh ra dãy thao tác chèn, xóa, thay có vị trí và ký tự rõ ràng, thực hiện lại được bằng một hàm độc lập và cho ra đúng Y.
- Kiểm tra dữ liệu vào trước khi chạy thuật toán: ký tự ngoài bảng chữ cái, xâu dài quá 30, dữ liệu không phải xâu đều bị từ chối.
- In được bảng quy hoạch động có nhãn hai xâu cho trường hợp độ dài không quá 6, phục vụ giải thích truy vết.

Sản phẩm nộp gồm mã nguồn Julia, bộ kiểm thử, log chạy thật, ảnh chụp trong VS Code và báo cáo này. Mã nguồn nằm trong tệp ZIP nộp kèm, có README mô tả lệnh chạy, và cũng đã công bố trên kho GitHub công khai (xem mục 4.9).

1.3. Phạm vi, đầu vào và đầu ra

1.3.1. Miền và quy ước đầu vào

Đầu vào là hai xâu ký tự, mỗi xâu dài từ 0 đến 30, chỉ chứa chữ cái Latin không dấu. Ký tự hoa và thường được coi là khác nhau, nên "Cat" và "CAT" có khoảng cách bằng 2 dù nhìn giống nhau. Xâu rỗng hợp lệ và được nhập bằng cách bấm Enter.

Chương trình nhận dữ liệu theo ba cách: truyền tham số vào hàm, chạy lệnh có sẵn trong main.jl, hoặc nhập từ bàn phím ở chế độ nhap. Không đọc tệp CSV, Excel hay bất kỳ nguồn dữ liệu ngoài nào.

Bảng 1.1. Đặc tả dữ liệu vào

Thành phần	Kiểu	Miền giá trị	Quy ước
X	xâu ký tự	$0 \le \|X\| \le 30$	chỉ A–Z, a–z; rỗng hợp lệ
Y	xâu ký tự	$0 \le \|Y\| \le 30$	chỉ A–Z, a–z; rỗng hợp lệ
Cách nhập	bàn phím / tham số	—	Enter cho xâu rỗng
Dữ liệu sai	—	—	từ chối kèm vị trí ký tự sai

1.3.2. Kết quả và trường hợp đặc biệt

Kết quả gồm ba phần: số thao tác nhỏ nhất, dãy thao tác theo thứ tự từ X đến Y, và danh sách các xâu sau từng bước để người đọc đối chiếu. Mỗi thao tác ghi rõ loại, vị trí và ký tự liên quan.

Bảng 1.2. Đặc tả kết quả đầu ra

Thành phần	Kiểu	Ý nghĩa	Ghi chú
khoang_cach	số nguyên	số thao tác ít nhất	bằng ô f[m, n]
buoc	danh sách	dãy thao tác theo thứ tự	chèn, xóa, thay
vi_tri	số nguyên	vị trí trên xâu hiện tại	đếm từ 1
lich_su	danh sách xâu	xâu sau mỗi bước	phần tử đầu là X

Các trường hợp đặc biệt đã xử lý: hai xâu cùng rỗng cho khoảng cách 0 và dãy rỗng; xâu đích rỗng thì xóa toàn bộ; hai xâu giống nhau cho khoảng cách 0 và không sinh thao tác thay; hai xâu không có ký tự chung cho khoảng cách bằng max(m, n). Dữ liệu sai được phân biệt với bài toán hợp lệ không có nghiệm: ở đây mọi cặp xâu hợp lệ đều có nghiệm, nên trường hợp duy nhất bị từ chối là dữ liệu vi phạm hợp đồng đầu vào.

1.4. Cách thực hiện và cấu trúc báo cáo

Bài làm đi theo bốn bước: đọc phiếu và giáo trình, làm tay một ví dụ nhỏ, cài đặt rồi kiểm thử, cuối cùng là chạy thật để lấy số liệu và ảnh. Bước làm tay quan trọng vì nó cho biết bảng phải trông như thế nào trước khi viết mã; nếu bỏ qua, rất dễ viết đúng công thức nhưng sai chiều chỉ số.

Bảng 1.3. Đối chiếu yêu cầu của phiếu và sản phẩm

Yêu cầu trong phiếu	Sản phẩm tương ứng	Vị trí
Tính khoảng cách nhỏ nhất	khoang_cach_sua	mục 3.2, 3.3
In dãy chèn, xóa, thay	truy_vet_sua	mục 3.3
Kiểm lại dãy thao tác	ap_dung_cac_buoc	mục 3.3, 3.4
Bảng có nhãn cho xâu ≤ 6	in_bang	mục 3.3
Kiểm dữ liệu vào	kiem_tra_dau_vao	mục 3.1
Chạy bàn phím hoặc tham số	main.jl với bảy lệnh	Phụ lục A

Năm chương tiếp theo lần lượt trình bày cơ sở lý thuyết, phân tích và cài đặt, kết quả thử nghiệm, rồi kết luận. Phụ lục giữ cây tệp, lệnh chạy và danh mục bằng chứng để người chấm đối chiếu nhanh.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Khái niệm và mô hình dữ liệu

Cho X gồm m ký tự và Y gồm n ký tự. Ký hiệu X[1..i] là i ký tự đầu của X, Y[1..j] là j ký tự đầu của Y. Đặt f[i, j] là số thao tác ít nhất để biến X[1..i] thành Y[1..j]. Hai tiền tố rỗng ứng với i = 0 hoặc j = 0.

Mỗi bước cuối của một cách biến đổi chỉ rơi vào một trong bốn trường hợp: xóa X[i], chèn Y[j], thay X[i] bằng Y[j], hoặc giữ nguyên khi X[i] = Y[j]. Từ đó có hệ thức truy hồi:

$$f[i, j] = \min(f[i-1, j] + 1,\ f[i, j-1] + 1,\ f[i-1, j-1] + (X[i] \neq Y[j] \ ?\ 1 : 0))$$

Trong đó f[i−1, j] + 1 là xóa X[i], f[i, j−1] + 1 là chèn Y[j], còn f[i−1, j−1] cộng 1 nếu hai ký tự khác nhau và cộng 0 nếu giống nhau. Điều kiện biên là f[0, j] = j và f[i, 0] = i, tương ứng chèn hoặc xóa toàn bộ.

Hệ thức này là nội dung bài toán khoảng cách chỉnh sửa trong giáo trình [1, chương 4], cũng là công thức mà Wagner và Fischer công bố năm 1974 [3].

Cách xây dựng này chính là quy hoạch động trên hai tiền tố, đúng hướng dẫn ở bước 1 đến bước 3 của phiếu. Giá trị cần tìm là f[m, n].

2.2. Cấu trúc dữ liệu sử dụng

2.2.1. Dữ liệu và thao tác chính

Bảng f là ma trận số nguyên kích thước (m+1) × (n+1). Với độ dài tối đa 30, bảng lớn nhất có 31 × 31 = 961 ô, rất nhỏ so với bộ nhớ máy, nên không cần nén.

Mỗi thao tác được lưu bằng một bản ghi BuocSua gồm bốn trường: loại thao tác (:giu, :thay, :xoa, :chen), vị trí, ký tự cũ và ký tự mới. Với thao tác xóa, trường ký tự mới nhận ký tự rỗng '\0'; với thao tác chèn, trường ký tự cũ nhận '\0'. Cách mã hóa này giữ mỗi bước trong một bản ghi cố định, dễ in và dễ kiểm. Giá trị :giu có trong kiểu dữ liệu nhưng hàm truy vết không sinh ra bước :giu, vì giữ nguyên ký tự không phải là thao tác.

Ngoài ra chương trình dùng một mảng ký tự tạm khi thực hiện lại dãy thao tác, vì xâu trong Julia là bất biến [4]; chèn và xóa được thực hiện trên mảng rồi đổi lại thành xâu để in.

2.2.2. Quy ước và điều kiện hợp lệ

Chỉ số của bảng tính từ 0 cho tiền tố rỗng, nhưng trong Julia mảng bắt đầu từ 1, nên chương trình lưu f[i+1, j+1] cho tiền tố dài i, j. Quy ước này được ghi rõ trong chú thích hàm để tránh nhầm khi đọc mã.

Vị trí thao tác đếm từ 1 và tính trên xâu tại thời điểm thao tác, không phải trên xâu gốc. Đây là quy ước quan trọng nhất của bài làm, được giải thích ở mục 2.3.3.

Bảng 2.1. Biến cấu trúc và điều kiện phải duy trì

Biến	Kiểu	Ý nghĩa	Điều kiện
X, Y	String	hai xâu vào	chỉ A–Z, a–z; dài 0–30
m, n	Int	độ dài X, Y	$0 \le m, n \le 30$
bang	Matrix{Int}	bảng quy hoạch động	$(m+1) \times (n+1)$ ô
f[i, j]	Int	chi phí cho hai tiền tố	luôn ≥ 0
buoc	Vector{BuocSua}	dãy thao tác	vị trí hợp lệ khi áp dụng
lich_su	Vector{String}	xâu sau mỗi bước	phần tử cuối bằng Y

2.3. Nguyên lý thuật toán

2.3.1. Ý tưởng quy hoạch động

Bài toán con là biến tiền tố X[1..i] thành tiền tố Y[1..j]. Nghiệm của bài toán lớn ghép từ nghiệm của bài toán nhỏ hơn theo một trong ba hướng: bỏ ký tự cuối của X, thêm ký tự cuối của Y, hoặc xử lý cặp ký tự cuối cùng nhau. Vì mọi cách biến đổi đều kết thúc bằng một trong ba hướng đó, lấy giá trị nhỏ nhất là đủ.

Thuật toán điền bảng theo thứ tự tăng dần của i và j, nên khi tính ô (i, j) thì ba ô cần dùng đã có sẵn. Không cần đệ quy và không cần lưu lại toàn bộ lời giải trung gian, chỉ cần bảng.

2.3.2. Khởi tạo và kết thúc

Hàng 0 được gán f[0, j] = j vì biến xâu rỗng thành j ký tự cần đúng j lần chèn. Cột 0 được gán f[i, 0] = i vì biến i ký tự thành xâu rỗng cần đúng i lần xóa. Hai dòng này là điều kiện dừng của truy hồi.

Vòng lặp chính chạy i từ 1 đến m và j từ 1 đến n. Kết thúc khi điền xong ô [m, n]; đó là khoảng cách cần tìm. Thuật toán luôn dừng vì số ô hữu hạn và mỗi vòng lặp tăng chỉ số.

2.3.3. Truy vết và quy ước vị trí

Truy vết bắt đầu từ ô [m, n] và đi ngược về [0, 0]. Tại mỗi ô, chương trình xét bốn khả năng theo thứ tự: nếu hai ký tự cuối bằng nhau và ô chéo bằng ô hiện tại thì giữ nguyên và lùi cả hai chỉ số; nếu không, so ô chéo cộng 1, ô trên cộng 1, ô trái cộng 1 để biết bước vừa rồi là thay, xóa hay chèn.

Các bước thu được theo thứ tự ngược, nên phải đảo lại trước khi in. Vị trí ghi vào mỗi bước được tính ngay tại ô đang xét, theo trạng thái xâu lúc thao tác được thực hiện: xóa X[i] khi đã có j ký tự đích đứng trước thì vị trí là j+1; thay X[i] bằng Y[j] thì vị trí là j; chèn Y[j] thì cũng là vị trí j.

Nhờ quy ước này, hàm ap_dung_cac_buoc chỉ cần chạy xuôi dãy thao tác và sửa trên xâu hiện tại, không phải quy đổi vị trí. Nếu ghi vị trí theo xâu gốc thì khi xâu đã dài ra hoặc ngắn đi, vị trí sẽ lệch và bước sau sai theo.

2.4. Cơ sở đánh giá thời gian và bộ nhớ

Phép toán chính là phép so sánh và cộng khi tính mỗi ô. Bảng có (m+1)(n+1) ô, mỗi ô xét ba khả năng nên thời gian là O(mn). Bộ nhớ phụ cũng O(mn) vì phải giữ cả bảng để truy vết; riêng dãy thao tác tốn O(m+n) và mảng ký tự tạm tốn O(m+n).

Nếu chỉ cần con số khoảng cách, có thể thu bộ nhớ về O(n) bằng cách giữ hai hàng. Bài này không dùng cách đó vì phiếu yêu cầu truy vết, mà muốn truy vết thì phải có toàn bộ bảng. Đây là đánh đổi có chủ ý, được ghi rõ trong báo cáo.

Với miền của phiếu, chi phí lớn nhất chỉ khoảng 961 ô nên thời gian chạy không đáng kể; phần lớn thời gian của chương trình là khởi động Julia và in kết quả.

CHƯƠNG 3. PHÂN TÍCH, THIẾT KẾ VÀ CÀI ĐẶT

3.1. Phân tích yêu cầu chương trình

3.1.1. Hợp đồng input/output

Chương trình gồm năm hàm chính. Ba hàm khoang_cach_sua, truy_vet_sua và ap_dung_cac_buoc là yêu cầu bắt buộc của phiếu; kiem_tra_dau_vao và in_bang là hai hàm hỗ trợ, phục vụ kiểm tra dữ liệu và in bảng. Bảng dưới ghi rõ tham số, giá trị trả về và lỗi có thể phát sinh.

Bảng 3.1. Hợp đồng của các hàm

Hàm	Tham số	Trả về	Lỗi
kiem_tra_dau_vao	X, Y	cặp xâu đã chuẩn hóa	ArgumentError khi sai hợp đồng
khoang_cach_sua	X, Y	(khoang_cach, bang)	không
truy_vet_sua	X, Y, bang	Vector{BuocSua}	ArgumentError nếu bảng sai cỡ
ap_dung_cac_buoc	X, buoc	(xau, lich_su)	ArgumentError nếu vị trí sai
in_bang	bang, X, Y	nothing (in ra màn hình)	ArgumentError nếu xâu dài hơn 6

3.1.2. Kiểm tra đầu vào

Hàm kiem_tra_dau_vao duyệt từng ký tự của cả hai xâu và từ chối khi gặp ký tự ngoài A–Z, a–z, khi độ dài vượt 30, hoặc khi tham số không phải xâu. Thông báo lỗi nêu tên xâu, ký tự sai và vị trí để người dùng sửa nhanh.

Ví dụ, nhập Y = "A1" sẽ nhận thông báo: Y có ký tự không hợp lệ '1' tại vị trí 2 (chỉ nhận A–Z, a–z). Chương trình dừng trước khi gọi thuật toán, nên không có trường hợp tính ra kết quả từ dữ liệu sai.

Cần phân biệt dữ liệu sai với bài toán hợp lệ nhưng kết quả đặc biệt. Hai xâu không có ký tự chung vẫn là dữ liệu hợp lệ và phải cho ra khoảng cách bằng max(m, n); chỉ ký tự ngoài bảng chữ cái hoặc độ dài vượt miền mới bị từ chối.

3.2. Thiết kế các hàm và dữ liệu

Mã nguồn chia thành ba tệp: thư viện DeTai.jl chứa toàn bộ thuật toán, main.jl là chương trình dòng lệnh, và test/kiem_thu.jl là bộ kiểm thử. Cách chia này giúp phần kiểm thử gọi trực tiếp hàm mà không phải mô phỏng nhập bàn phím.

Bảng 3.2. Tổ chức các hàm và tệp

Tệp	Hàm	Trách nhiệm
src/DeTai.jl	kiem_tra_dau_vao	kiểm hợp đồng đầu vào
src/DeTai.jl	khoang_cach_sua	điền bảng quy hoạch động
src/DeTai.jl	truy_vet_sua	truy ngược và đảo dãy thao tác
src/DeTai.jl	ap_dung_cac_buoc	thực hiện lại dãy, trả xâu cuối
src/DeTai.jl	in_bang	in bảng có nhãn cho xâu ngắn
main.jl	chay_vd1, chay_vd2, chay_phieu	chạy các ca có sẵn
main.jl	chay_nhap, chay_bang, chay_cap, chay_tatca	nhập bàn phím, in bảng, chạy tất cả
test/kiem_thu.jl	14 testset	kiểm 13 ca và bất biến

Thiết kế dữ liệu đã trình bày ở mục 2.2: bảng số nguyên, bản ghi BuocSua và mảng ký tự tạm. Không dùng thư viện ngoài, không dùng cấu trúc phức tạp hơn mức cần thiết, đúng tinh thần của phiếu.

Bảng 3.3. Tiền điều kiện và hậu điều kiện

Hàm	Tiền điều kiện	Hậu điều kiện
khoang_cach_sua	X, Y là xâu hợp lệ	bang[end, end] bằng khoảng cách nhỏ nhất
truy_vet_sua	bảng đúng cỡ (m+1)×(n+1)	dãy bước có đúng f[m, n] thao tác
ap_dung_cac_buoc	vị trí nằm trong xâu hiện tại	xâu cuối bằng Y
in_bang	độ dài hai xâu không quá 6	bảng in đủ (m+1)(n+1) ô

3.3. Thuật toán và mã nguồn chính

3.3.1. Giả mã theo cài đặt

Ba thủ tục dưới đây viết theo đúng tên biến trong mã nguồn, chỉ khác một chỗ: bảng được gọi tắt là f, còn trong mã nguồn tên biến là bang.

```
khoang_cach_sua(X, Y):
    m ← |X|; n ← |Y|
    tạo bảng f kích thước (m+1) × (n+1), toàn số 0
    for i ← 1..m: f[i, 0] ← i
    for j ← 1..n: f[0, j] ← j
    for i ← 1..m:
        for j ← 1..n:
            thay ← f[i-1, j-1] + (X[i] = Y[j] ? 0 : 1)
            xoa  ← f[i-1, j] + 1
            chen ← f[i, j-1] + 1
            f[i, j] ← min(thay, xoa, chen)
    return (f[m, n], f)

truy_vet_sua(X, Y, f):
    i ← m; j ← n; buoc ← rỗng
```

```
    while i > 0 hoặc j > 0:
        nếu X[i] = Y[j] và f[i, j] = f[i-1, j-1]:
            i ← i-1; j ← j-1          # giữ nguyên
        ngược lại nếu f[i, j] = f[i-1, j-1] + 1:
            ghi thay (vị trí j, X[i] → Y[j]); i ← i-1; j ← j-1
        ngược lại nếu f[i, j] = f[i-1, j] + 1:
            ghi xóa (vị trí j+1, ký tự X[i]); i ← i-1
        ngược lại:
            ghi chèn (vị trí j, ký tự Y[j]); j ← j-1
    đảo buoc
    return buoc

ap_dung_cac_buoc(X, buoc):
    s ← mảng ký tự của X; lich_su ← [X]
    với mỗi b trong buoc:
        kiểm vị trí và ký tự cũ của b trên s
        thay / xóa / chèn tương ứng
        thêm bản sao của s vào lich_su
    return (xâu(s), lich_su)
```

3.3.2. Đoạn mã và giải thích

Đoạn quyết định là vòng lặp điền bảng. Biến thay, xoa, chen lần lượt ứng với ba hướng của hệ thức truy hồi; dùng min của ba giá trị là đủ vì chi phí mọi thao tác đều bằng 1.

```
for i in 1:m, j in 1:n
    thay = bang[i, j] + (X[i] == Y[j] ? 0 : 1)
    xoa = bang[i, j + 1] + 1
    chen = bang[i + 1, j] + 1
    bang[i + 1, j + 1] = min(thay, xoa, chen)
end
```

Chú ý cách đánh chỉ số: bang[i+1, j+1] ứng với tiền tố dài i, j; vì vậy ô trên là bang[i, j+1] và ô trái là bang[i+1, j]. Nếu viết nhầm thành bang[i, j] cho ô trên thì kết quả vẫn chạy nhưng sai ở các ca có chèn hoặc xóa.

Phần truy vết kiểm tra bốn trường hợp theo đúng thứ tự đã nêu. Thứ tự này không ảnh hưởng đến con số khoảng cách, nhưng quyết định dãy thao tác được chọn khi có nhiều phương án cùng chi phí: đặt giữ nguyên trước để ưu tiên dãy ngắn và dễ đọc, sau đó mới đến thay, xóa và chèn.

3.3.3. Xử lý tình huống đặc biệt

Khi một xâu rỗng, bảng chỉ còn một hàng hoặc một cột và vòng lặp chính không chạy; kết quả lấy trực tiếp từ điều kiện biên. Khi hai xâu giống nhau, mọi bước đều rơi vào nhánh giữ nguyên nên dãy thao tác rỗng và khoảng cách bằng 0.

Khi độ dài bằng 30, bảng có 961 ô, vẫn nằm trong miền cho phép. Hàm in_bang từ chối in nếu xâu dài hơn 6 để tránh bảng quá rộng trên màn hình; đây là giới hạn của phần tùy chọn, không phải giới hạn của thuật toán.

Hàm ap_dung_cac_buoc kiểm cả vị trí lẫn ký tự trước khi sửa. Nếu dãy thao tác bị sai, chương trình báo lỗi ngay tại bước sai thay vì âm thầm tạo ra xâu khác Y.

3.4. Lần vết ví dụ nhỏ

Ca truy vết thứ nhất là ví dụ 1 của phiếu: X = "CAT", Y = "CUT". Bảng f có kích thước 4 × 4, hàng và cột 0 là điều kiện biên.

Bảng 3.4. Bảng f cho CAT → CUT

i \ j	ε	C	U	T
ε	0	1	2	3
C	1	0	1	2
A	2	1	1	2
T	3	2	2	1

Ô cuối bằng 1 nên chỉ cần một thao tác. Truy ngược: tại ô [3,3], X[3] = Y[3] = 'T' và ô chéo [2,2] cũng bằng 1 nên giữ T; tại ô [2,2], X[2] = 'A' khác Y[2] = 'U' và ô chéo [1,1] bằng 0 nên phát sinh một lần thay; ô [1,1] có X[1] = Y[1] = 'C' nên giữ tiếp. Dãy thao tác chỉ có một bước.

Bảng 3.5. Lần vết CAT → CUT

Bước	Xâu hiện tại	Thao tác
0	CAT	bắt đầu
1	CUT	thay vị trí 2: 'A' → 'U'

Ca thứ hai dài hơn để thấy cả ba loại thao tác: X = "SUNDAY", Y = "SATURDAY". Khoảng cách bằng 3, gồm hai lần chèn và một lần thay.

Bảng 3.6. Lần vết SUNDAY → SATURDAY

Bước	Xâu hiện tại	Thao tác
0	SUNDAY	bắt đầu
1	SAUNDAY	chèn vị trí 2: 'A'
2	SATUNDAY	chèn vị trí 3: 'T'
3	SATURDAY	thay vị trí 5: 'N' → 'R'

Hai lần chèn làm xâu dài dần, nên nếu vị trí ghi theo xâu gốc thì bước thay cuối sẽ nhắm sai chỗ. Đây chính là lý do bài làm chọn quy ước vị trí trên xâu hiện tại và kiểm lại bằng hàm áp dụng: sau bước 3, xâu nhận được đúng bằng SATURDAY.

3.5. Giải thích tính đúng và điều kiện dừng

3.5.1. Điều kiện khởi đầu và duy trì

Bất biến của vòng lặp điền bảng: sau khi tính xong ô (i, j), giá trị f[i, j] bằng số thao tác ít nhất biến X[1..i] thành Y[1..j]. Khởi đầu đúng vì hàng 0 và cột 0 biểu diễn chèn hoặc xóa toàn bộ, mỗi ký tự đúng một thao tác.

Bước duy trì: một cách biến đổi tối ưu X[1..i] thành Y[1..j] phải kết thúc bằng xóa X[i], chèn Y[j], hoặc xử lý cặp X[i], Y[j] cùng nhau. Ba trường hợp này cho ba ứng viên trong công thức; lấy nhỏ nhất nên f[i, j] không lớn hơn phương án tối ưu, đồng thời mỗi ứng viên đều tương ứng một cách biến đổi hợp lệ nên không nhỏ hơn. Do đó đẳng thức đúng.

Với truy vết, bất biến là tại ô (i, j) đang xét, phần đã ghi có thể biến X[1..i] thành Y[1..j] với đúng f[i, j] thao tác. Mỗi bước lùi về một ô có giá trị nhỏ hơn đúng 1 (hoặc bằng 0 khi giữ nguyên), nên tổng số thao tác ghi được bằng f[m, n].

3.5.2. Kết thúc và output

Vòng lặp truy vết giảm i hoặc j ở mọi bước, mà i, j không âm, nên dừng sau tối đa m + n bước. Khi dừng, mọi ký tự của X đã được xử lý và mọi ký tự của Y đã được sinh ra, nên xâu cuối bằng Y.

Chương trình không tin vào lập luận suông: sau khi truy vết, hàm ap_dung_cac_buoc thực hiện lại toàn bộ dãy và trả về xâu cuối; phép so sánh xâu cuối với Y nằm ở hàm gọi, tức main.jl và bộ kiểm thử. Nếu sai, chương trình báo lỗi ngay tại bước vi phạm. Nhờ vậy kết quả in ra vừa có con số vừa có bằng chứng tự kiểm.

3.6. Phân tích độ phức tạp

Bảng 3.7. Chi phí của từng pha trong bản cài

Pha	Phép toán chính	Thời gian	Bộ nhớ phụ
Kiểm tra dữ liệu	duyệt ký tự	O(m + n)	O(m + n)
Điền bảng	3 phép so sánh mỗi ô	O(mn)	O(mn)
Truy vết	lùi từng ô	O(m + n)	O(m + n)
Áp dụng và in	mỗi bước sửa một ký tự	O((m + n)²)	O(m + n)
Toàn chương trình	—	O((m + n)²)	O(mn)

Với miền của phiếu, m, n ≤ 30 nên chi phí lớn nhất khoảng 961 phép xét ô. Riêng pha áp dụng dãy thao tác phải chèn hoặc xóa trên mảng ký tự nên mỗi bước tốn O(m + n), cả pha là O((m + n)²); khi m, n cùng bậc thì đây vẫn nằm trong O(mn), còn khi lệch nhau thì phải tính theo (m + n)². Con số đo được rất nhỏ, và phần lớn thời gian chạy thực tế là khởi động môi trường Julia.

Bộ nhớ O(mn) là bắt buộc nếu muốn truy vết. Nếu chỉ cần con số, có thể dùng hai hàng và bộ nhớ giảm còn O(n); hướng phát triển ở chương 5 sẽ nói thêm về cách chia để trị của Hirschberg, vừa giữ được truy vết vừa giảm bộ nhớ.

CHƯƠNG 4. KẾT QUẢ THỬ NGHIỆM VÀ NHẬN XÉT

4.1. Môi trường và kế hoạch kiểm thử

Bài làm chạy trên Windows 11, Julia 1.13.1 (chỉ thư viện chuẩn), trình soạn thảo VS Code 1.142.0 với terminal tích hợp. Mã nguồn đặt tại thư mục N24DCDT065_DT056_NguyenNguNhut, bộ kiểm thử tại test/kiem_thu.jl.

Kế hoạch kiểm thử gồm ba lớp. Lớp thứ nhất là 13 ca độc lập: hai ví dụ của phiếu, năm ca bắt buộc, hai ca tự thiết kế, ba ca biên và một ca dữ liệu sai. Lớp thứ hai là kiểm bất biến trên 400 cặp xâu ngẫu nhiên. Lớp thứ ba là khảo sát ba quy mô để đối chiếu với phân tích độ phức tạp.

Kỳ vọng của từng ca được tính trước khi chạy, bằng cách lập bảng tay hoặc suy ra từ tính chất của bài toán. Ví dụ, hai xâu không có ký tự chung thì mỗi vị trí phải thay hoặc chèn, nên khoảng cách bằng max(m, n). Không có ca nào được sửa kỳ vọng sau khi chạy để biến sai thành đúng.

4.2. Bảng kết quả 13 ca kiểm thử

Bảng 4.1 đối chiếu năm ca bắt buộc của phiếu với mã ca trong bộ kiểm thử. Hai ví dụ của phiếu cũng có mặt nên tổng số ca chức năng là bảy.

Bảng 4.1. Ánh xạ năm ca kiểm trên phiếu

Ca trên phiếu	Nội dung	Mã ca	Kết quả
1	hai xâu rỗng, khoảng cách 0	P01	đạt
2	rỗng → AB, hai lần chèn	P02	đạt
3	ABC → ABC, không thay	P03	đạt
4	AB → BA, hai lần thay	P04	đạt
5	ABC → AC, xóa B	P05	đạt
Ví dụ 1	CAT → CUT, thay giữa	VD01	đạt
Ví dụ 2	A → rỗng, xóa	VD02	đạt

Bảng 4.2 thiết kế 13 ca độc lập. Cột kỳ vọng là con số tính trước; cột mục đích ghi điều mà ca đó kiểm.

Bảng 4.2. Thiết kế 13 ca kiểm thử độc lập

Mã	X	Y	Kỳ vọng	Mục đích
VD01	CAT	CUT	1	ví dụ 1, thay ký tự giữa
VD02	A	(rỗng)	1	ví dụ 2, xóa ký tự duy nhất
P01	(rỗng)	(rỗng)	0	hai xâu rỗng
P02	(rỗng)	AB	2	chèn toàn bộ
P03	ABC	ABC	0	giữ nguyên, không thay
P04	AB	BA	2	không dùng đổi chỗ
P05	ABC	AC	1	xóa ký tự giữa
TK01	SUNDAY	SATURDAY	3	trộn chèn và thay
TK02	kitten	sitting	3	chữ thường, trộn ba loại
B01	A	B	1	biên một ký tự khác nhau
B02	ABCDEFGHIJKLMNOPQRSTUVWXYZABCD	ABCDEFGHIJKLMNOPQRSTUVWXYZABCE	1	biên độ dài tối đa
B03	AAAA	AA	2	biên xóa lặp
LOI01	A×31; A1; A B	—	báo lỗi	validator từ chối

Bảng 4.3 ghi kết quả thật của lần chạy cuối. Cột minh chứng chỉ ra ảnh hoặc log tương ứng để người chấm đối chiếu.

Bảng 4.3. Kết quả thật và vị trí minh chứng

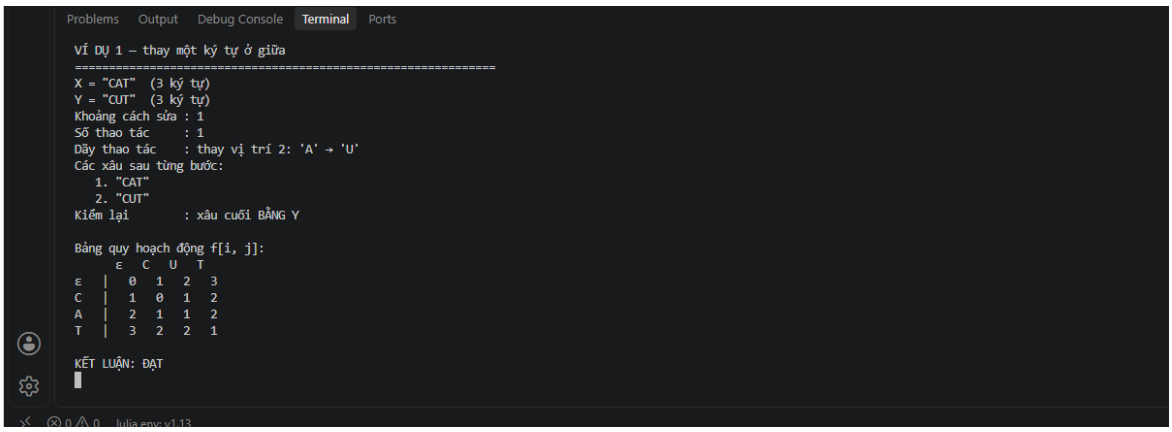
Mã	Khoảng cách	Dãy thao tác	Kết luận	Minh chứng
VD01	1	thay 2: A→U	đạt	A01, vd1.log
VD02	1	xóa 1: A	đạt	A02, vd2.log
P01	0	rỗng	đạt	phieu.log
P02	2	chèn 1: A; chèn 2: B	đạt	phieu.log, A03P
P03	0	rỗng	đạt	phieu.log, A03P
P04	2	thay 1: A→B; thay 2: B→A	đạt	phieu.log, A03P
P05	1	xóa 2: B	đạt	phieu.log, A03P
TK01	3	chèn 2: A; chèn 3: T; thay 5: N→R	đạt	A03, kiem_thu.log
TK02	3	thay 1: k→s; thay 5: e→i; chèn 7: g	đạt	A04, kiem_thu.log
B01	1	thay 1: A→B	đạt	A05, kiem_thu.log
B02	1	thay 30: D→E	đạt	A06, kiem_thu.log
B03	2	xóa 1: A; xóa 1: A	đạt	A07, kiem_thu.log
LOI01	—	từ chối dữ liệu	đạt	A08, kiem_thu.log

Bảng 4.1 đánh số ca 1–5 theo đúng thứ tự trình bày trong phiếu; khi đối chiếu chỉ cần dùng cột mã ca (P01–P05).

Lượt chạy cả năm ca bằng lệnh julia main.jl phieu được đối chiếu qua tệp docs/logs/phieu.log. Ảnh chụp lượt chạy này (mã A03P) chỉ thấy phần cuối cửa sổ nên để ở Phụ lục B dưới dạng ảnh phụ, không tính vào mười một ảnh bắt buộc.

4.3. Hai ví dụ trên phiếu

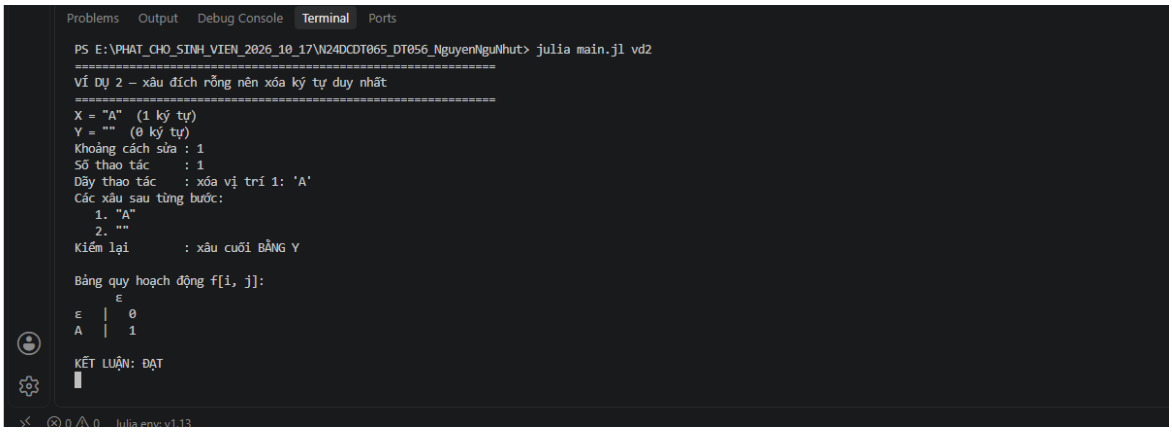
Hình 4.1 là màn hình VS Code khi chạy lệnh julia main.jl vd1. Bảng f in ra khớp với bảng tính tay ở mục 3.4, ô cuối bằng 1, dãy thao tác chỉ có một lần thay và dòng kiểm lại xác nhận xâu cuối bằng Y.



Hình 4.1. A01 — Ví dụ 1: CAT → CUT, khoảng cách 1

Điểm đáng chú ý là dòng "Kiểm lại: xâu cuối BẰNG Y". Dòng này do chính chương trình sinh ra sau khi áp dụng dãy thao tác, không phải câu ghi tay, nên nếu thuật toán sai thì ảnh sẽ thể hiện ngay.

Hình 4.2 là ví dụ 2 với xâu đích rỗng. Bảng chỉ còn một cột, hàm truy vết đi theo nhánh xóa đến khi hết xâu. Kết quả khoảng cách 1 và xâu cuối rỗng, đúng kỳ vọng.



Hình 4.2. A02 — Ví dụ 2: A → xâu rỗng, khoảng cách 1

Hai ví dụ này cũng cho thấy chương trình không truy cập ngoài xâu khi độ dài bằng 0, điều dễ sai nếu viết vòng lặp theo chỉ số từ 1.

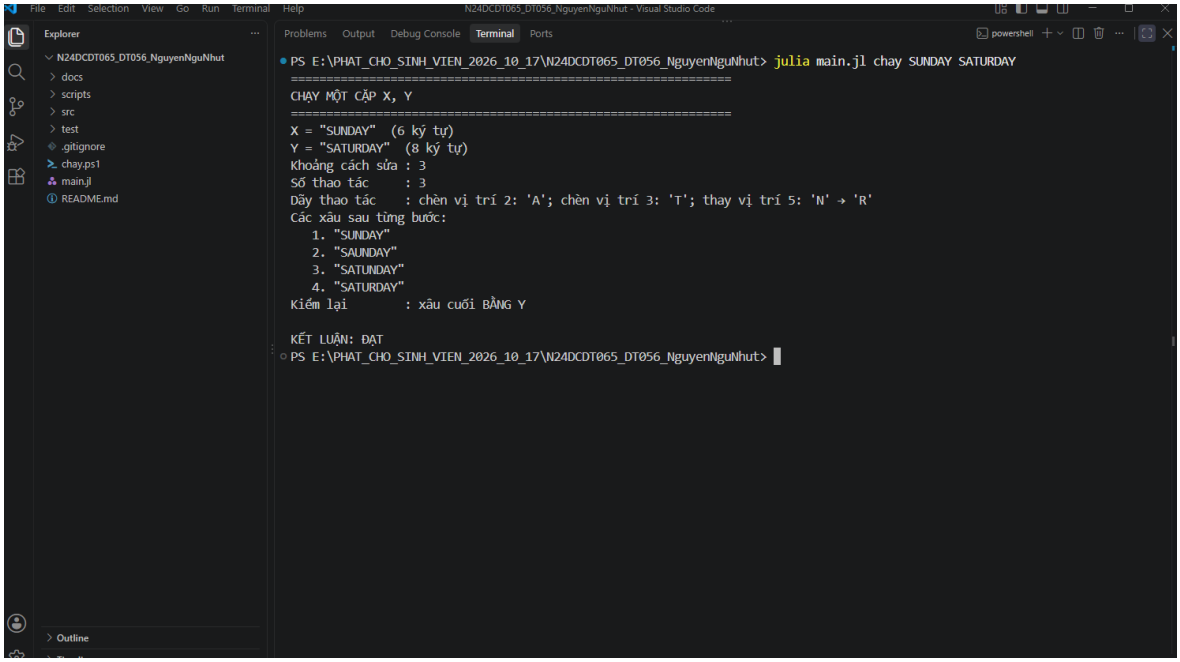
4.4. Hai ca sinh viên tự thiết kế

Ca TK01 dùng X = "SUNDAY", Y = "SATURDAY", khoảng cách tính tay bằng 3 vì phải chèn A, chèn T và thay N thành R. Kết quả chạy cho đúng 3 bước theo thứ tự đó, xâu cuối bằng SATURDAY. Ca này kiểm nhánh chèn liên tiếp, khác với ví dụ 1 chỉ có thay.

Ca TK02 dùng X = "kitten", Y = "sitting", khoảng cách 3 gồm thay k thành s, thay e thành i và chèn g ở cuối. Ca này kiểm chữ thường và kiểm trường hợp ba loại thao tác cùng xuất hiện trong một dãy. Cả hai ca đều đạt và dãy thao tác áp dụng lại cho đúng xâu đích.

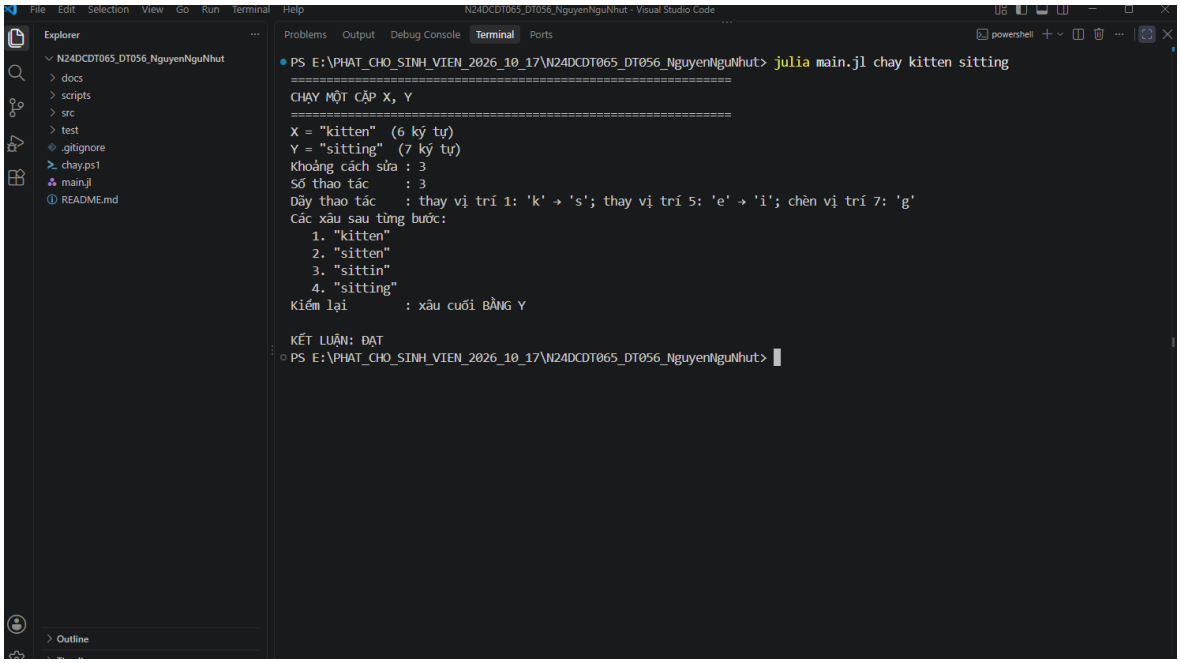
Hai ca được chọn có diễn tiến khác nhau: TK01 làm xâu dài thêm ở giữa, TK02 vừa thay ở đầu vừa chèn ở cuối. Nhờ vậy nếu hàm áp dụng hiểu sai vị trí, ít nhất một trong hai ca sẽ sai.

Hình 4.3 chụp ca TK01 trong VS Code: dòng lệnh julia main.jl chay SUNDAY SATURDAY, khoảng cách 3, dãy chèn A, chèn T rồi thay N thành R, dòng kiểm lại xác nhận xâu cuối bằng SATURDAY.



Hình 4.3. A03 — Ca TK01: SUNDAY → SATURDAY, khoảng cách 3

Hình 4.4 là ca TK02 với chữ thường: thay k thành s, thay e thành i và chèn g ở cuối, xâu cuối bằng sitting.



Hình 4.4. A04 — Ca TK02: kitten → sitting, khoảng cách 3

4.5. Ba ca biên phù hợp với đề tài

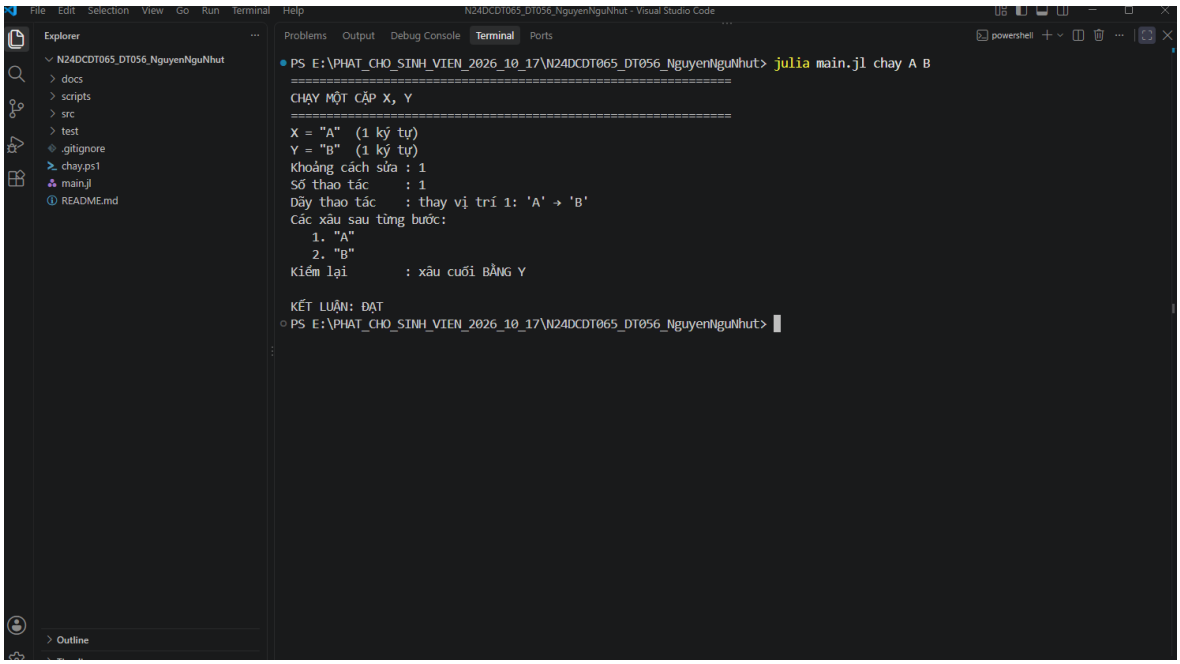
Ca B01 dùng hai xâu một ký tự khác nhau, A và B, kỳ vọng 1. Đây là biên nhỏ nhất của bài toán có thao tác, kiểm trường hợp bảng chỉ có 2 × 2 ô.

Ca B02 dùng hai xâu dài đúng 30 ký tự, chỉ khác ký tự cuối, kỳ vọng 1. Ca này kiểm biên trên của miền và kiểm bảng 31 × 31 ô, đồng thời kiểm việc không tràn khi độ dài bằng giới hạn.

Ca B03 dùng AAAA → AA, kỳ vọng 2. Ca này kiểm nhánh xóa lặp và kiểm trường hợp nhiều dãy tối ưu khác nhau cùng tồn tại; chương trình chỉ cần trả một dãy đúng, không bắt buộc trùng với dãy trong báo cáo.

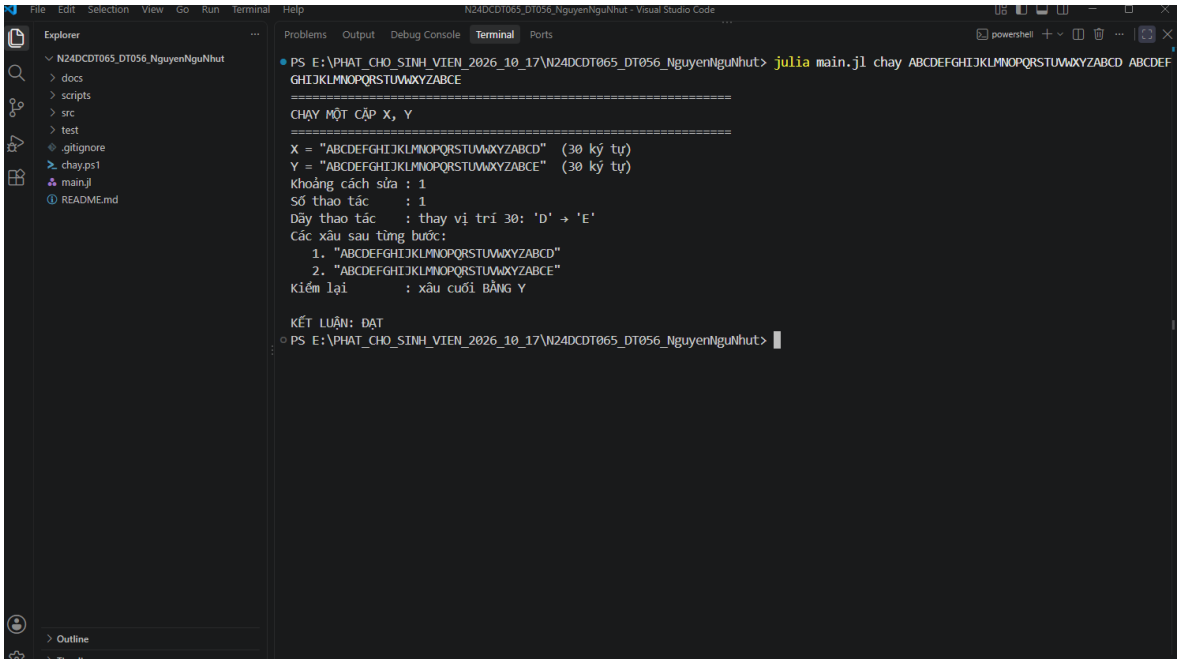
Hai ca tự thiết kế và ba ca biên đều đã có ảnh chụp riêng: A03 và A04 ở mục 4.4, A05 đến A07 ở mục 4.5. Mỗi ảnh thấy cả dòng lệnh lẫn kết quả, khớp với Bảng 4.2, Bảng 4.3 và tệp log kiem_thu.log.

Hình 4.5 là ca biên nhỏ nhất A → B: bảng chỉ có 2 × 2 ô và đúng một lần thay.



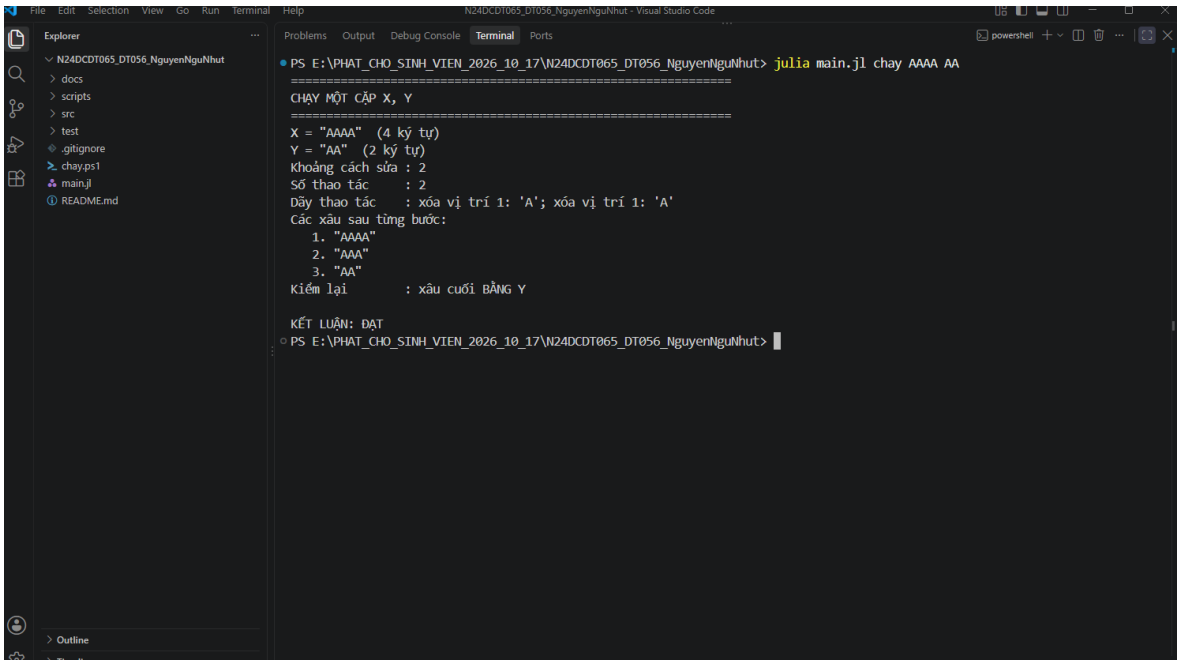
Hình 4.5. A05 — Ca B01: A → B, khoảng cách 1

Hình 4.6 là ca độ dài 30: hai xâu chỉ khác ký tự cuối nên khoảng cách 1, thay vị trí 30 từ D thành E.



Hình 4.6. A06 — Ca B02: hai xâu 30 ký tự, khoảng cách 1

Hình 4.7 là ca xóa lặp AAAA → AA: khoảng cách 2 và cả hai bước đều xóa ở vị trí 1, đúng như dòng lệnh in ra.



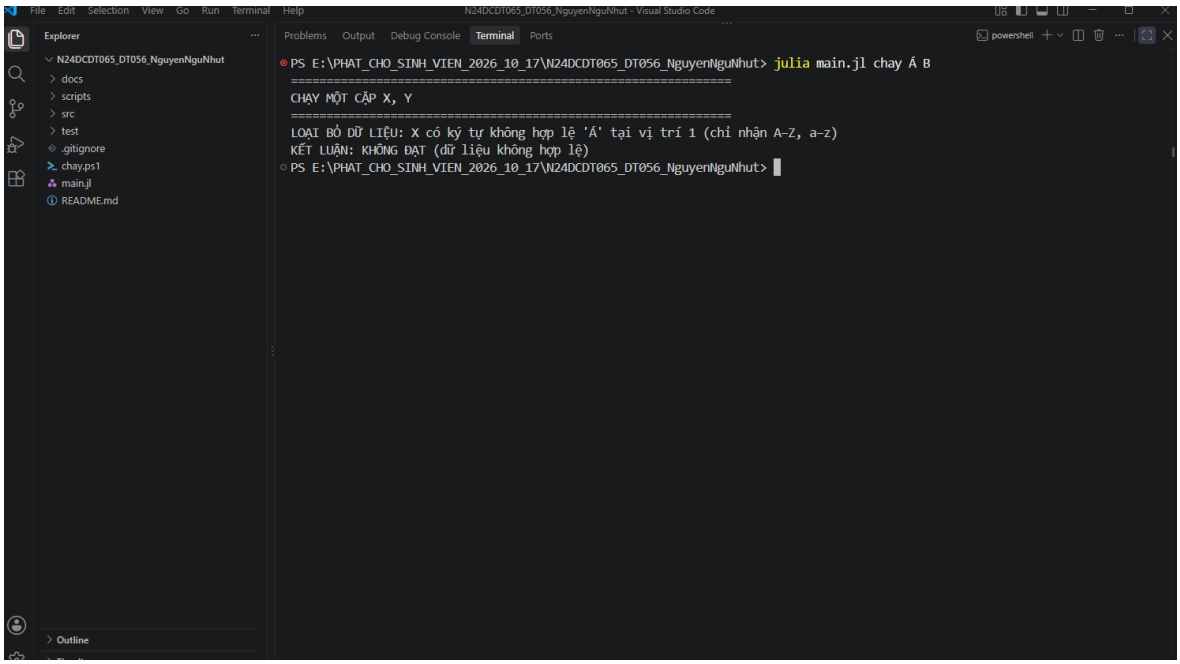
Hình 4.7. A07 — Ca B03: AAAA → AA, khoảng cách 2

4.6. Kiểm tra input không hợp lệ — LOI01

Ca LOI01 đưa vào năm dữ liệu sai: xâu dài 31 ký tự, xâu chứa chữ số, xâu chứa dấu cách, tham số không phải xâu và tham số nothing. Cả năm đều bị hàm kiểm tra từ chối bằng ArgumentError với thông báo nêu rõ vị trí. Chương trình không chạy thuật toán trên dữ liệu sai, nên không có kết quả rác.

Về phía chấp nhận, điều kiện là "không quá 30" chứ không phải "nhỏ hơn 30": hai xâu dài đúng 30 vẫn hợp lệ, và ca B02 ở mục 4.5 đã dùng đúng biên này.

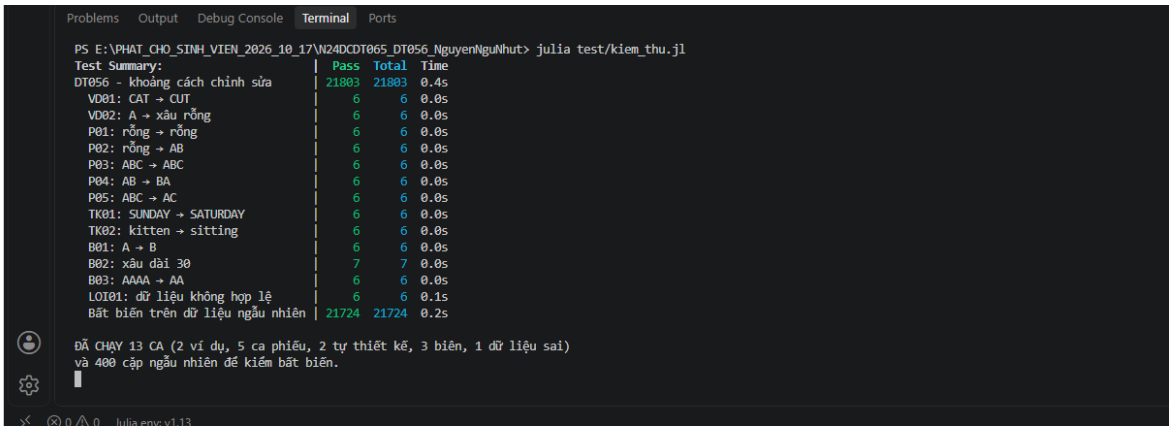
Hình 4.8 là ca dữ liệu sai: chương trình dừng trước khi tính, báo ký tự không hợp lệ 1 tại vị trí 2 và kết luận không đạt.



Hình 4.8. A08 — Ca LOI01: dữ liệu sai bị từ chối

4.7. Kết quả toàn bộ kiểm thử

Hình 4.9 là tổng kết một lần chạy toàn bộ bộ kiểm thử bằng lệnh julia test/kiem_thu.jl. Mười ba ca đều đạt; tổng số phép kiểm là 21.803, trong đó 79 phép kiểm theo ca và 21.724 phép kiểm bất biến trên 400 cặp xâu ngẫu nhiên.



Hình 4.9. A09 — Tổng kết 13 ca, 21.803 phép kiểm đạt

Số phép kiểm lớn hơn nhiều so với số ca vì đơn vị đếm là phép kiểm, không phải cặp. Mỗi cặp có bốn phép kiểm mức cặp (áp dụng ra đúng Y, số thao tác bằng ô cuối, khoảng cách đối xứng, khoảng cách không vượt m + n) và ba phép kiểm cho mỗi ô của bảng (không lệch quá 1 so với ba ô liền trước). Cặp dài 8 × 8 cho tới 196 phép, nên 400 cặp cộng lại thành 21.724.

4.8. Khảo sát ba quy mô

Ba quy mô được chọn trong miền của phiếu: m = n = 10, 20 và 30. Mỗi quy mô dùng một cặp xâu sinh ngẫu nhiên trên bảng chữ A–H với hạt giống cố định, nên lần chạy sau lặp lại đúng kết quả. Thời gian là trung vị của năm đợt đo sau một lần chạy nóng; số lần gọi mỗi đợt là 2.000 ở quy mô 10, 500 ở quy mô 20 và 200 ở quy mô 30.

Bảng 4.4. Khảo sát ba quy mô

Quy mô	m = n	Số ô bảng	Số bước truy vết	Thời gian một lần gọi
Q1	10	121	8	1,02 µs
Q2	20	441	16	3,42 µs
Q3	30	961	22	7,41 µs

Số ô tăng 3,64 lần rồi 2,18 lần; thời gian tăng 3,35 lần rồi 2,16 lần, cùng xu hướng với số ô. Ở quy mô nhỏ phép đo còn nhiễu nên báo cáo chỉ lấy thời gian làm tham khảo; căn cứ chính là số ô và số bước truy vết, khớp với phân tích O(mn) và O(m + n).

Bộ nhớ chỉ được phân tích theo lý thuyết, chưa đo bằng công cụ. Bảng lớn nhất trong miền của phiếu có 31 × 31 = 961 số nguyên, chỉ vài chục KB, quá nhỏ để đo chính xác bằng công cụ của hệ điều hành; vì vậy báo cáo ghi O(mn) cho bảng và nói rõ đây là phần chưa có số đo thực nghiệm.

4.9. Kho mã và trạng thái công bố

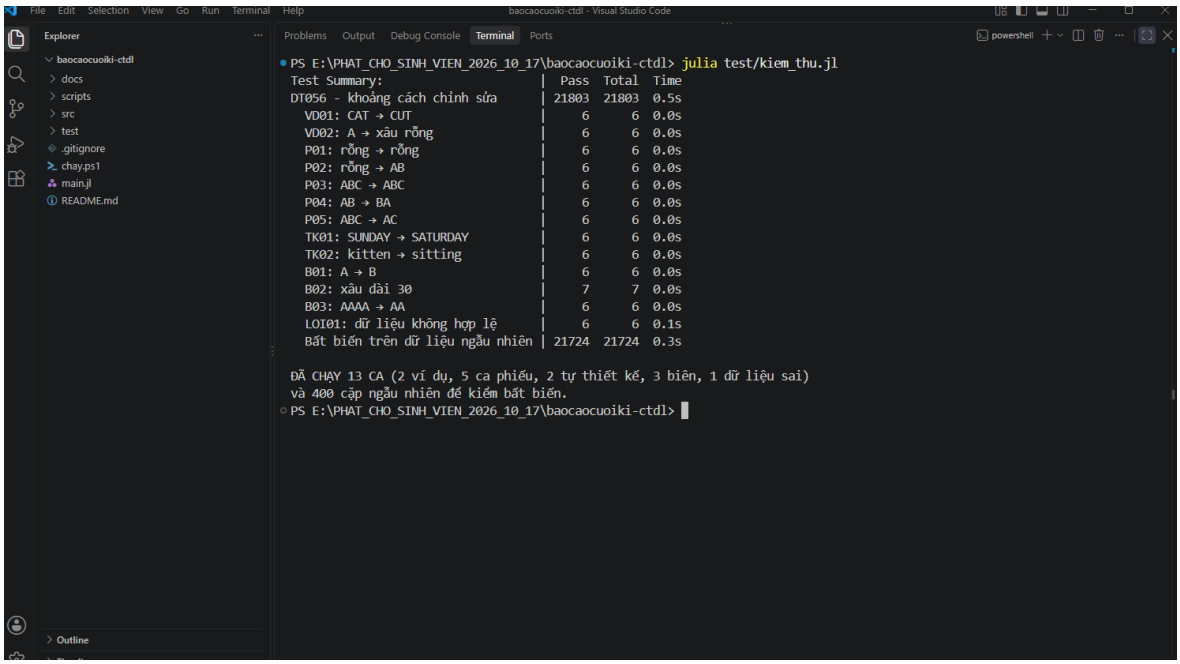
Mã nguồn được lưu trong kho GitHub công khai ở địa chỉ dưới đây. Kho đã nhận bản nộp: nhánh main ở commit b77199d, và commit này chỉ thêm mười ảnh minh chứng so với commit fcb4d2e, không sửa mã nguồn, bộ kiểm thử hay dữ liệu vào. Hai đường dẫn tới src/DeTai.jl và test/kiem_thu.jl tại commit đã cho mở được bằng trình duyệt ẩn danh.

Bảng 4.5. Kho mã và trạng thái

Hạng mục	Giá trị
Kho mã	https://github.com/tlnhut456-a11y/baocaocuoiki-ctdl
Trạng thái kho	công khai, nhánh main có 2 commit
Commit C (mã, test, log)	fcb4d2e3b7f8c9a60bfd2cf89fa871ff2c4766e2
Commit S (bản nộp)	b77199d2bf845f23152a09eaf5d907e3e5eee51f
Nhánh	main
Tệp thuật toán chính	src/DeTai.jl
Tệp kiểm thử	test/kiem_thu.jl

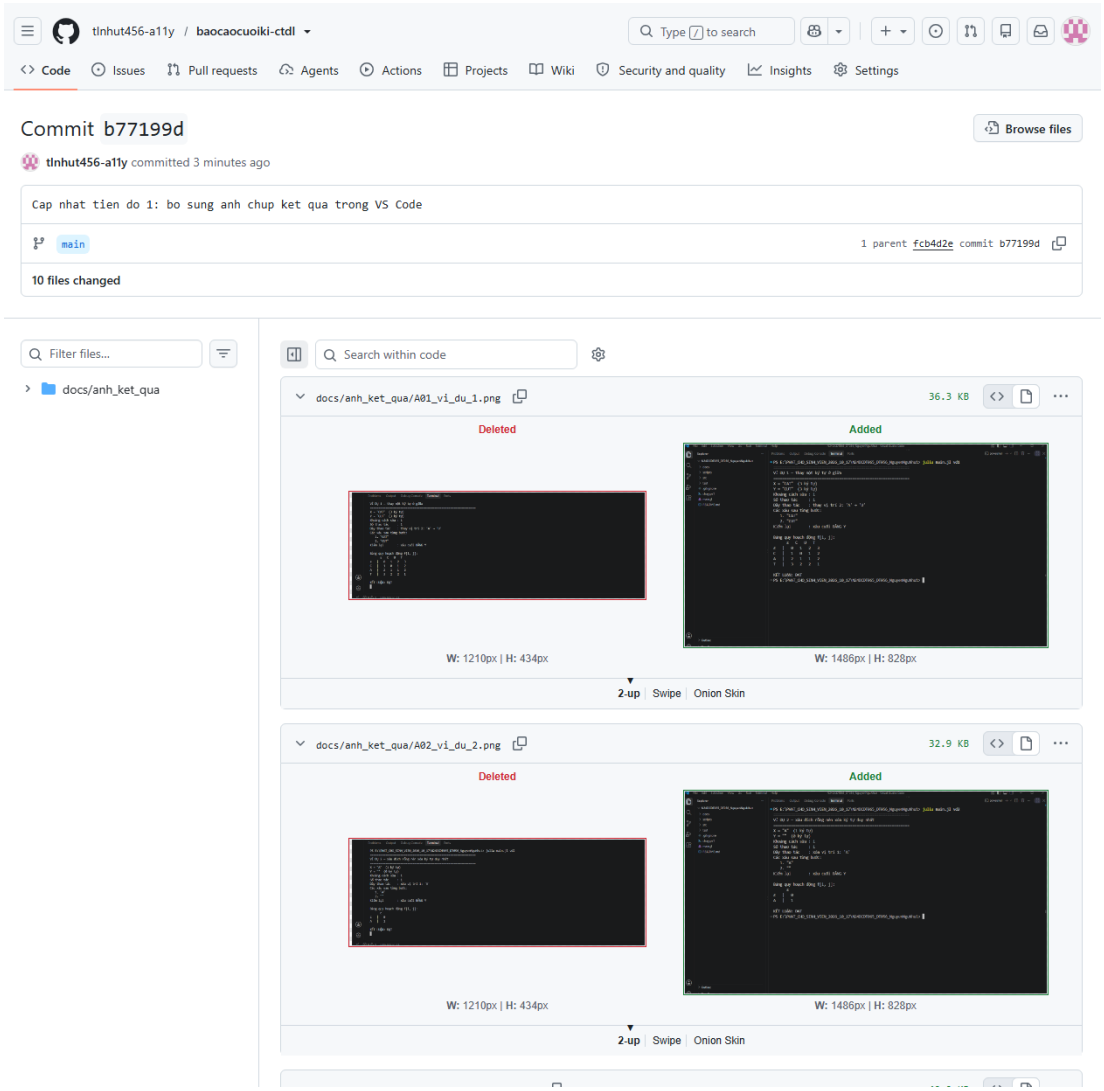
Hạng mục	Giá trị
Ảnh A10, A11	Hình 4.10 và Hình 4.11

Hình 4.10 là màn hình VS Code sau khi clone kho về thư mục mới và chạy lại julia test/kiem_thu.jl từ commit b77199d: mười ba ca đều đạt, tổng 21.803/21.803 phép kiểm, khớp với kết quả ở mục 4.7. Lần chạy này dùng bản clone chứ không dùng thư mục làm việc ban đầu, nên nếu gói thiếu tệp thì sẽ lỗi ngay. Hình 4.11 là trang commit trên GitHub: SHA, thông điệp commit, nhánh main và danh sách mười tệp ảnh đã thay.



Hình 4.10. A10 — Clone kho ở commit S rồi chạy lại bộ kiểm thử

Hình 4.11 là trang commit b77199d trên GitHub; phần dưới trang là so sánh ảnh cũ và ảnh mới của từng tệp.



Hình 4.11. A11 — Trang commit b77199d trên GitHub

CHƯƠNG 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1. Mức hoàn thành so với phiếu

Bảng 5.1 đối chiếu từng yêu cầu của phiếu với sản phẩm và bằng chứng tương ứng.

Bảng 5.1. Mức hoàn thành và bằng chứng

Yêu cầu	Mức	Bằng chứng
Tính khoảng cách nhỏ nhất	đầy đủ	mục 3.3, 13 ca, A01–A09
In dãy thao tác	đầy đủ	mục 3.3, 3.4, A01–A09
Kiểm lại dãy thao tác	đầy đủ	ap_dung_cac_buoc, dòng kiểm lại
Kiểm dữ liệu vào	đầy đủ	mục 3.1.2, ca LOI01
Bảng có nhãn cho xâu ≤ 6	đầy đủ	in_bang, A01, A02
Năm ca của phiếu	đầy đủ	Bảng 4.1, phieu.log, A03P (Phụ lục B)
Mười một ảnh minh chứng	đầy đủ	A01–A11 (11/11)
Kho mã và ảnh A10, A11	đầy đủ	mục 4.9, kho công khai, 2 commit

5.2. Giới hạn và bài học

Bản cài chỉ xử lý chữ cái Latin không dấu và độ dài tối đa 30 theo đúng miền của phiếu. Chi phí ba thao tác đều bằng 1 nên không dùng được cho bài toán có trọng số khác nhau. Bộ nhớ O(mn) là bắt buộc vì phải giữ bảng để truy vết.

Bài học lớn nhất là chuyện vị trí thao tác. Lần đầu viết, em ghi vị trí theo xâu gốc và ca P02 cho ra dãy chèn sai vị trí dù khoảng cách vẫn đúng 2. Bảng lần vết giúp phát hiện: sau lần chèn đầu, xâu đã dài thêm một ký tự nên vị trí chèn thứ hai phải là 2 chứ không phải 1. Từ đó em đổi quy ước sang vị trí trên xâu hiện tại và thêm hàm áp dụng để tự kiểm.

Bài học thứ hai là không nên tin vào con số khoảng cách. Có những dãy thao tác cho đúng khoảng cách nhưng áp dụng lại không ra Y; chỉ khi chạy lại dãy mới phát hiện. Vì vậy báo cáo luôn kèm dòng kiểm lại trong ảnh chụp.

5.3. Hướng phát triển

Hướng thứ nhất là mở rộng sang khoảng cách chỉnh sửa có trọng số, trong đó chi phí chèn, xóa, thay khác nhau. Công thức giữ nguyên, chỉ thay ba hằng số 1 bằng ba trọng số; phần khó là chọn trọng số hợp lý cho từng ứng dụng.

Hướng thứ hai là giảm bộ nhớ bằng chia để trị theo Hirschberg [5]: tính hai nửa bảng rồi đệ quy, vẫn truy vết được mà chỉ tốn O(n) bộ nhớ. Cách này phù hợp nếu miền độ dài tăng lên hàng nghìn.

Hướng thứ ba là dùng kết quả khoảng cách để làm công cụ so sánh văn bản: hiển thị các đoạn giống nhau, tô màu chỗ thêm bớt. Phần này mới là đề xuất, chưa cài đặt trong bản nộp.

TÀI LIỆU THAM KHẢO

[1] Giáo trình Cấu trúc dữ liệu và Giải thuật INT1306, Chương 4, Bài tập chương 4 câu 35 (bài toán khoảng cách chỉnh sửa), trang in 112, trang PDF 124.

[2] Levenshtein V. I. (1966). Binary codes capable of correcting deletions, insertions, and reversals. Soviet Physics Doklady, 10(8), 707–710.

[3] Wagner R. A., Fischer M. J. (1974). The string-to-string correction problem. Journal of the ACM, 21(1), 168–173.

[4] Julia Language 1.13 Documentation, mục Strings và Arrays, https://docs.julialang.org/en/v1/ (truy cập ngày 20/10/2026).

[5] Hirschberg D. S. (1975). A linear space algorithm for computing maximal common subsequences. Communications of the ACM, 18(6), 341–343.

PHỤ LỤC A. TỔ CHỨC MÃ VÀ HƯỚNG DẪN CHẠY

```
N24DCDT065_DT056_NguyenNguNhut/
├── main.jl
├── src/DeTai.jl
├── test/kiem_thu.jl
├── scripts/thong_ke.jl
├── scripts/chup_anh_vscode.ps1
├── chay.ps1
├── README.md
└── docs/
    ├── PHIEU_GOC_DT056.txt
    ├── MoPhong_KhoangCachSua_DT056.html
    ├── logs/            8 tệp log
    └── anh_ket_qua/     12 ảnh: A01–A11 và A03P
```

Yêu cầu môi trường: Julia 1.13.1, không cần gói ngoài. Mở terminal tại thư mục gốc của bài rồi chạy một trong các lệnh sau.

Bảng PL.1. Các lệnh chạy

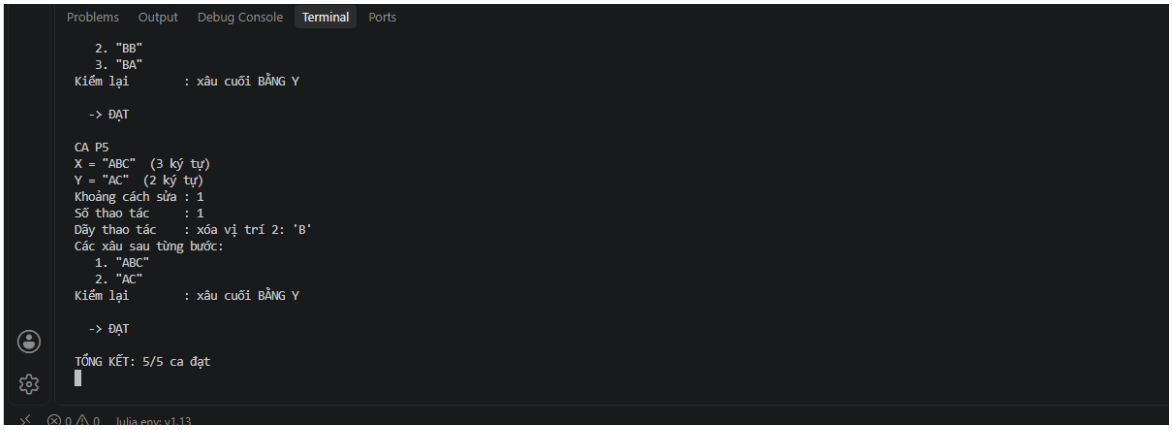
Lệnh	Việc làm
julia main.jl vd1	chạy ví dụ 1
julia main.jl vd2	chạy ví dụ 2
julia main.jl phieu	chạy năm ca của phiếu
julia main.jl nhap	nhập X, Y từ bàn phím
julia main.jl bang CAT CUT	in bảng quy hoạch động
julia main.jl chay SUNDAY SATURDAY	chạy một cặp X, Y bất kỳ
julia test/kiem_thu.jl	chạy 13 ca kiểm thử
julia scripts/thong_ke.jl	khảo sát ba quy mô
powershell -ExecutionPolicy Bypass -File .\chay.ps1	chạy một lượt toàn bộ

Trang mô phỏng docs/MoPhong_KhoangCachSua_DT056.html mở trực tiếp bằng trình duyệt, không cần mạng. Trang cho chọn ví dụ có sẵn hoặc nhập X, Y, điền bảng theo từng ô, tô ba ô dùng để tính, sau đó tô đường truy vết và liệt kê dãy thao tác.

PHỤ LỤC B. DANH MỤC BẰNG CHỨNG

Bảng PL.2. Ảnh và log tương ứng

Mã	Tệp	Nội dung
A01	docs/anh_ket_qua/A01_vi_du_1.png	ví dụ 1, bảng f, dãy thao tác
A02	docs/anh_ket_qua/A02_vi_du_2.png	ví dụ 2, xâu đích rỗng
A03	docs/anh_ket_qua/A03_TK01.png	ca TK01, SUNDAY → SATURDAY
A04	docs/anh_ket_qua/A04_TK02.png	ca TK02, kitten → sitting
A05	docs/anh_ket_qua/A05_B01.png	ca B01, A → B
A06	docs/anh_ket_qua/A06_B02.png	ca B02, hai xâu 30 ký tự
A07	docs/anh_ket_qua/A07_B03.png	ca B03, AAAA → AA
A08	docs/anh_ket_qua/A08_LOI01.png	ca LOI01, dữ liệu sai bị từ chối
A09	docs/anh_ket_qua/A09_tong_ket_kiem_thu.png	13 ca, 21.803 phép kiểm
A10	docs/anh_ket_qua/A10_clone_chay_lai.png	clone kho, chạy lại bộ kiểm thử
A11	docs/anh_ket_qua/A11_github_commit.png	trang commit b77199d trên GitHub
A03P (phụ)	docs/anh_ket_qua/A03P_nam_ca_phieu.png	năm ca phiếu, phần cuối cửa sổ
—	docs/logs/vd1.log, vd2.log	log hai ví dụ
—	docs/logs/phieu.log	log năm ca phiếu
—	docs/logs/kiem_thu.log	log bộ kiểm thử
—	docs/logs/thong_ke.log	log khảo sát ba quy mô
—	docs/logs/bang_cat_cut.log	log in bảng CAT → CUT
—	docs/logs/nhap_hop_le.log, nhap_sai.log	log nhập bàn phím



Hình PL.1. A03P — Ảnh phụ: phần cuối lượt chạy julia main.jl phieu (ca P02–P05, tổng kết 5/5 đạt)

Mười một ảnh A01–A11 đã có; A03P là ảnh phụ. Các tệp ảnh nằm trong gói ZIP nộp kèm, thư mục docs/anh_ket_qua.

PHỤ LỤC C. TỰ KIỂM TRƯỚC KHI NỘP

- Đúng MSSV N24DCDT065, lớp D24CQDT01-N, mã đề DT056.
- Đã cài đủ ba hàm bắt buộc và phần tùy chọn in bảng.
- 13 ca kiểm thử độc lập, chạy thật 21.803 phép kiểm, không sửa kỳ vọng.
- Có bảng truy vết, giải thích tính đúng, phân tích O(mn) và khảo sát ba quy mô.
- Mười một ảnh A01–A11 đã có trong báo cáo; A03P để ở Phụ lục B dưới dạng ảnh phụ.
- Báo cáo đã xóa hết câu hướng dẫn và ngoặc vuông của khung.
- Word và PDF cùng một phiên bản, mục lục và số trang đã cập nhật.

PHỤ LỤC D. HƯỚNG DẪN NỘP BÀI

Bốn tệp nộp dùng chung tiền tố N24DCDT065_DT056_NguyenNguNhut, họ tên không dấu, không khoảng trắng.

Bảng PL.3. Bốn tệp cần nộp

Tệp	Nội dung
..._BaoCao.docx	báo cáo cá nhân đã điền đầy đủ
..._BaoCao.pdf	bản PDF xuất từ đúng Word cuối
..._MaNguon.zip	mã nguồn, test, README, ảnh gốc
..._ThongTinNop.txt	thông tin nộp, URL kho, SHA

Trước khi nộp cần giải nén ZIP vào thư mục mới và chạy lại chay.ps1 để chắc bài chạy được độc lập. SHA S và trang commit đã ghi ở mục 4.9 và trong ThongTinNop.txt, kèm hai ảnh A10, A11.